Assignment 16.5

Name:V.Sravani
Ht.No:2303A51923
Batch: 23

Task 1 – Employee Management Database
Task:
Create a database schema for managing employee records in an organization.
Instructions:
• Tables: Employees, Departments, Salaries.
• Use AI to define constraints such as NOT NULL and UNIQUE.
• Generate queries:
o List all employees in a specific department.
o Retrieve salary details of an employee.
o Find the highest-paid employee in each department.
Expected Output:
• Schema and SQL queries demonstrating joins and constraints.

Code:

```sql
-- Create an employee table
CREATE TABLE employee1 (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone_number VARCHAR(20),
    hire_date DATE NOT NULL,
    job_title VARCHAR(50) NOT NULL,
    salary DECIMAL(10, 2) NOT NULL,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES department(id)
);
-- Create a departments table
CREATE TABLE department1 (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(100) NOT NULL UNIQUE,
    manager_id INT,
    FOREIGN KEY (manager_id) REFERENCES employee1(id)
);
--Create salaries table
CREATE TABLE salary1 (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    employee_id INTEGER NOT NULL,
    amount DECIMAL(10, 2) NOT NULL,
    effective_date DATE NOT NULL,
    FOREIGN KEY (employee_id) REFERENCES employee1(id)
);
-- Insert sample data into department table
INSERT INTO department1 (name, manager_id) VALUES
('Human Resources', NULL),
('Engineering', NULL),
('Marketing', NULL),
('Finance', NULL),
('Sales', NULL);
-- Insert sample data into employee table
INSERT INTO employee1 (first_name, last_name, email, phone_number, hire_date, job_title, salary, department_id) VALUES
('John', 'Doe', 'john.doe@example.com', '123-456-7890', '2022-01-01', 'Software Engineer', 75000.00, 2),
('Jane', 'Smith', 'jane.smith@example.com', '098-765-4321', '2022-01-01', 'Sales Associate', 50000.00, 3),
('Emily', 'Johnson', 'emily.johnson@example.com', '555-555-5555', '2022-01-01', 'Marketing Specialist', 55000.00, 4),
('Michael', 'Brown', 'michael.brown@example.com', '555-123-4567', '2022-01-01', 'Finance Analyst', 60000.00, 5),
('Sarah', 'Davis', 'sarah.davis@example.com', '555-987-6543', '2022-01-01', 'Human Resources Specialist', 50000.00, 1);
```

```sql
-- Insert sample data into salary table
INSERT INTO salary1 (employee_id, amount, effective_date) VALUES
(1, 75000.00, '2022-01-01'),
(2, 50000.00, '2022-01-01'),
(3, 55000.00, '2022-01-01'),
(4, 60000.00, '2022-01-01'),
(5, 50000.00, '2022-01-01');
-- List all employees in a specific department
SELECT e.first_name, e.last_name, d.name AS department
FROM employee1 e
JOIN department1 d ON e.department_id = d.id
WHERE d.name = 'Engineering';
-- Retreive the salary details of an employee
SELECT e.first_name, e.last_name, s.amount, s.effective_date
FROM employee1 e
JOIN salary1 s ON e.id = s.employee_id
WHERE e.email = 'john.doe@example.com';
-- display the highest paid employee in each department
SELECT d.name AS department, e.first_name, e.last_name, MAX(s.amount) AS highest_salary
FROM employee1 e
JOIN department1 d ON e.department_id = d.id
JOIN salary1 s ON e.id = s.employee_id
GROUP BY d.name;
```

Output:

SQL ▾ < 1 / 1 > 1 - 1 of 1

first_name	last_name	department
John	Doe	Engineering

SQL ▾ < 1 / 1 > 1 - 1 of 1

first_name	last_name	amount	effective_date
John	Doe	75000	2022-01-01

SQL ▾ < 1 / 1 > 1 - 5 of 5

department	first_name	last_name	highest_salary
Engineering	John	Doe	75000
Finance	Emily	Johnson	55000
Human Resources	Sarah	Davis	50000
Marketing	Jane	Smith	50000
Sales	Michael	Brown	60000

Task 2 – Online Course Platform Database

Task:

Design a database for an online learning platform.

Instructions:

• Tables: Users, Courses, Lessons, Enrollments.

• Define relationships between users and courses.

• Generate queries:

o List all courses enrolled by a user.

o Count number of learners per course.

o Retrieve lesson details for a course.

Expected Output:

• Relational schema with SQL queries using joins.

Code:

C: > Users > PC > Downloads > aiac > onlineLearningPlatform.sql

```sql
-- Create a users table
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password TEXT NOT NULL,
    role TEXT CHECK(role IN ('student', 'instructor')) NOT NULL,
    registration_date TEXT NOT NULL
);
-- Create a courses table
CREATE TABLE courses (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT,
    instructor_id INT,
    FOREIGN KEY (instructor_id) REFERENCES users(id)
);
-- Create a lessons table
CREATE TABLE lessons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_id INT,
    title TEXT NOT NULL,
    content TEXT,
    FOREIGN KEY (course_id) REFERENCES courses(id)
);
-- Create an enrollments table
CREATE TABLE enrollments (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INT,
    course_id INT,
    enrollment_date TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (course_id) REFERENCES courses(id)
);
-- Insert sample data into users table
INSERT INTO users (first_name, last_name, email, password, role, registration_date) VALUES
('Alice', 'Johnson', 'alice.johnson@example.com', 'password123', 'student', '2022-01-01'),
('Bob', 'Smith', 'bob.smith@example.com', 'password123', 'student', '2022-01-01'),
('Charlie', 'Brown', 'charlie.brown@example.com', 'password123', 'instructor', '2022-01-01'),
('David', 'Wilson', 'david.wilson@example.com', 'password123', 'student', '2022-01-01');
```

```sql
-- Insert sample data into courses table
INSERT INTO courses (title, description, instructor_id) VALUES
('Introduction to Programming', 'Learn the basics of programming using Python.', 3),
('Data Structures and Algorithms', 'Explore fundamental data structures and algorithms.', 3),
('Web Development', 'Build modern web applications using HTML, CSS, and JavaScript.', 3);
-- Insert sample data into lessons table
INSERT INTO lessons (course_id, title, content) VALUES
(1, 'Variables and Data Types', 'In this lesson, you will learn about variables and data types in Python.'),
(1, 'Control Structures', 'In this lesson, you will learn about control structures in Python.'),
(2, 'Arrays', 'In this lesson, you will learn about arrays in data structures.'),
(2, 'Linked Lists', 'In this lesson, you will learn about linked lists in data structures.'),
(3, 'HTML Basics', 'In this lesson, you will learn the basics of HTML.'),
(3, 'CSS Basics', 'In this lesson, you will learn the basics of CSS.'),
(3, 'JavaScript Basics', 'In this lesson, you will learn the basics of JavaScript.');
-- Insert sample data into enrollments table
INSERT INTO enrollments (user_id, course_id, enrollment_date) VALUES
(1, 1, '2022-01-01'),
(1, 2, '2022-01-01'),
(2, 1, '2022-01-01'),
(3, 1, '2022-01-01'),
(4, 3, '2022-01-01');
--List all courses enrolled by a user
SELECT c.title, e.enrollment_date
FROM courses c
JOIN enrollments e ON c.id = e.course_id
WHERE e.user_id = 1;
--Count number of learners enrolled per course
SELECT c.title, COUNT(e.user_id) AS enrolled_students
FROM courses c
JOIN enrollments e ON c.id = e.course_id
GROUP BY c.title;
-- Retrieve lesson details for a course
SELECT l.title, l.content
FROM lessons l
JOIN courses c ON l.course_id = c.id
WHERE c.title = 'Introduction to Programming';
```

Output:

SQL ▾ 1 / 1 1 - 2 of 2

title	enrollment_date
Introduction to Programming	2022-01-01
Data Structures and Algorithms	2022-01-01

SQL ▾ 1 / 1 1 - 3 of 3

title	enrolled_students
Data Structures and Algorithms	1
Introduction to Programming	3
Web Development	1

SQL ▾ 1 / 1 1 - 2 of 2

title	content
Variables and Data Types	In this lesson, you will learn about variables and data types in Python.
Control Structures	In this lesson, you will learn about control structures in Python.

Task 3 – Banking Transaction Database

Task:

Create a schema for a banking transaction management system.

Instructions:

• Tables: Customers, Accounts, Transactions.

• Use AI to suggest constraints for transaction validity.

• Generate queries:

o Retrieve transaction history for an account.

o Calculate total deposits and withdrawals.

o Find accounts with balance below a threshold.

Expected Output:

• Secure database schema and analytical SQL queries.

Code:

```sql
-- Create a customers table
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone_number VARCHAR(20),
    registration_date DATE NOT NULL
);
-- Create an accounts table
CREATE TABLE accounts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER NOT NULL,
    account_type VARCHAR(20) CHECK(account_type IN ('savings', 'checking')) NOT NULL,
    balance DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);
-- Create a transactions table
CREATE TABLE transactions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    account_id INTEGER NOT NULL,
    transaction_type VARCHAR(20) CHECK(transaction_type IN ('deposit', 'withdrawal')) NOT NULL,
    amount DECIMAL(10, 2) NOT NULL,
    transaction_date DATE NOT NULL,
    FOREIGN KEY (account_id) REFERENCES accounts(id)
);
-- Insert sample data into customers table
INSERT INTO customers (first_name, last_name, email, phone_number, registration_date) VALUES
('Alice', 'Johnson', 'alice.johnson@example.com', '555-1234', '2022-01-01'),
('Bob', 'Smith', 'bob.smith@example.com', '555-5678', '2022-01-01'),
('Charlie', 'Brown', 'charlie.brown@example.com', '555-9012', '2022-01-01'),
('David', 'Wilson', 'david.wilson@example.com', '555-3456', '2022-01-01');
-- Insert sample data into accounts table
INSERT INTO accounts (customer_id, account_type, balance) VALUES
(1, 'savings', 1000.00),
(1, 'checking', 500.00),
(2, 'savings', 2000.00),
(3, 'checking', 1500.00),
(4, 'savings', 3000.00);
-- Insert sample data into transactions table
```

```sql
-- Insert sample data into transactions table
INSERT INTO transactions (account_id, transaction_type, amount, transaction_date) VALUES
(1, 'deposit', 500.00, '2022-01-10'),
(1, 'withdrawal', 200.00, '2022-01-15'),
(2, 'deposit', 1000.00, '2022-01-12'),
(3, 'withdrawal', 300.00, '2022-01-20'),
(4, 'deposit', 1500.00, '2022-01-18');
-- Retrieve transaction history for an account
SELECT t.transaction_type, t.amount, t.transaction_date
FROM transactions t
JOIN accounts a ON t.account_id = a.id
WHERE a.customer_id = 1 AND a.account_type = 'savings';
-- Calculate the total deposits and withdrawals
SELECT
    SUM(CASE WHEN transaction_type = 'deposit' THEN amount ELSE 0 END) AS total_deposits,
    SUM(CASE WHEN transaction_type = 'withdrawal' THEN amount ELSE 0 END) AS total_withdrawals
FROM transactions t
JOIN accounts a ON t.account_id = a.id
WHERE a.customer_id = 1 AND a.account_type = 'savings';
-- Find accounts with the balance below a threshold
SELECT a.id, a.account_type, a.balance
FROM accounts a
WHERE a.balance < 1000.00;
```

Output:

SQL ▼ < 1 / 1 > 1 - 2 of 2

transaction_type	amount	transaction_date
deposit	500	2022-01-10
withdrawal	200	2022-01-15

SQL ▼ < 1 / 1 > 1 - 1 of 1

total_deposits	total_withdrawals
500	200

SQL ▼ < 1 / 1 > 1 - 1 of 1

id	account_type	balance
2	checking	500

Task 4 – Airline Reservation Database

Task:

Design a database for an airline reservation system.

Instructions:

• Tables: Flights, Passengers, Tickets, Bookings.

• Define relationships and seat allocation logic.

• Generate queries:

o List all passengers on a specific flight.

o Find available seats for a flight.

o Count total bookings per flight.

Expected Output:

• Schema with SQL queries using joins and counts.

Code :

```sql
--Create flight table
CREATE TABLE flight (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    flight_number VARCHAR(10) NOT NULL UNIQUE,
    departure_city VARCHAR(50) NOT NULL,
    arrival_city VARCHAR(50) NOT NULL,
    departure_time DATETIME NOT NULL,
    arrival_time DATETIME NOT NULL,
    price DECIMAL(10, 2) NOT NULL
);
--Create passengers table
CREATE TABLE passengers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone_number VARCHAR(20)
);
--create tickets table
CREATE TABLE tickets (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    flight_id INTEGER NOT NULL,
    passenger_id INTEGER NOT NULL,
    seat_number VARCHAR(5) NOT NULL,
    booking_date DATETIME NOT NULL,
    FOREIGN KEY (flight_id) REFERENCES flight(id),
    FOREIGN KEY (passenger_id) REFERENCES passengers(id)
);
--Create bookings table
CREATE TABLE bookings (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    passenger_id INTEGER NOT NULL,
    booking_date DATETIME NOT NULL,
    total_price DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (passenger_id) REFERENCES passengers(id)
);
--Insert sample data into flight table
INSERT INTO flight (flight_number, departure_city, arrival_city, departure_time, arrival_time, price) VALUES
('AA101', 'New York', 'Los Angeles', '2022-01-01 08:00:00', '2022-01-01 11:00:00', 300.00),
('BA202', 'London', 'Paris', '2022-01-02 09:00:00', '2022-01-02 10:30:00', 150.00),
('CA303', 'Beijing', 'Shanghai', '2022-01-03 10:00:00', '2022-01-03 12:00:00', 200.00);
```

```sql
--Insert sample data into passengers table
INSERT INTO passengers (first_name, last_name, email, phone_number) VALUES
('Alice', 'Johnson', 'alice.johnson@example.com', '555-123-4567'),
('Bob', 'Smith', 'bob.smith@example.com', '555-987-6543'),
('Charlie', 'Brown', 'charlie.brown@example.com', '555-555-5555');
--Insert sample data into tickets table
INSERT INTO tickets (flight_id, passenger_id, seat_number, booking_date) VALUES
(1, 1, '12A', '2022-01-01 09:00:00'),
(1, 2, '12B', '2022-01-01 09:30:00'),
(2, 3, '5C', '2022-01-02 10:00:00');
--Insert sample data into bookings table
INSERT INTO bookings (passenger_id, booking_date, total_price) VALUES
(1, '2022-01-01 09:00:00', 300.00),
(2, '2022-01-01 09:30:00', 300.00),
(3, '2022-01-02 10:00:00', 150.00);
--List all passengers on a specific flight
SELECT p.first_name, p.last_name, t.seat_number
FROM passengers p
JOIN tickets t ON p.id = t.passenger_id
WHERE t.flight_id = 1;
--Find available seats for a flight
SELECT f.seat_number
FROM tickets f
WHERE f.flight_id = 1
AND f.seat_number NOT IN (
    SELECT t.seat_number
    FROM tickets t
    WHERE t.flight_id = 1
);
-- Count total bookings per flight
SELECT f.flight_number, COUNT(t.id) AS total_bookings
FROM flight f
LEFT JOIN tickets t ON f.id = t.flight_id
GROUP BY f.flight_number;
```

Output:

SQL ▾ 1 / 1 1 - 2 of 2

first_name	last_name	seat_number
Alice	Johnson	12A
Bob	Smith	12B

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 3 of 3

flight_number	total_bookings
AA101	2
BA202	1
CA303	0

Task 5 – Transport Management Database

Task:

Design a database for transport route management.

Instructions:

• Tables: Vehicles, Routes, Drivers, Assignments.

• Use AI to suggest indexing strategies.

• Generate queries:

o List vehicles assigned to a route.

o Retrieve driver details for a vehicle.

o Count trips per route.

Expected Output:

• Schema and optimized SQL queries.

Code:

```sql
--Create vehicles table
CREATE TABLE vehicles (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    make VARCHAR(50) NOT NULL,
    model VARCHAR(50) NOT NULL,
    year INTEGER NOT NULL,
    license_plate VARCHAR(20) NOT NULL UNIQUE
);
--create routes table
CREATE TABLE routes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    origin VARCHAR(50) NOT NULL,
    destination VARCHAR(50) NOT NULL,
    distance DECIMAL(10, 2) NOT NULL
);
--create drivers table
CREATE TABLE drivers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    license_number VARCHAR(20) NOT NULL UNIQUE,
    phone_number VARCHAR(20)
);
--create assignments table
CREATE TABLE assignments (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    vehicle_id INTEGER NOT NULL,
    driver_id INTEGER NOT NULL,
    route_id INTEGER NOT NULL,
    assignment_date DATE NOT NULL,
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(id),
    FOREIGN KEY (driver_id) REFERENCES drivers(id),
    FOREIGN KEY (route_id) REFERENCES routes(id)
);
--Insert sample data into vehicles table
INSERT INTO vehicles (make, model, year, license_plate) VALUES
('Toyota', 'Camry', 2020, 'ABC123'),
('Honda', 'Civic', 2019, 'XYZ789'),
('Ford', 'F-150', 2021, 'DEF456');
--Insert sample data into routes table
INSERT INTO routes (origin, destination, distance) VALUES
('New York', 'Los Angeles', 2800.00),
('Chicago', 'Houston', 1080.00),
('Miami', 'Atlanta', 660.00);
```

```sql
--Insert sample data into drivers table
INSERT INTO drivers (first_name, last_name, license_number, phone_number) VALUES
('John', 'Doe', 'D1234567', '555-123-4567'),
('Jane', 'Smith', 'D2345678', '555-987-6543'),
('Emily', 'Johnson', 'D3456789', '555-555-5555');
--Insert sample data into assignments table
INSERT INTO assignments (vehicle_id, driver_id, route_id, assignment_date) VALUES
(1, 1, 1, '2022-01-01'),
(2, 2, 2, '2022-01-02'),
(3, 3, 3, '2022-01-03');
--List vehicles assigned to a route.
SELECT v.make, v.model, v.year, v.license_plate, r.origin, r.destination
FROM vehicles v
JOIN assignments a ON v.id = a.vehicle_id
JOIN routes r ON a.route_id = r.id
WHERE r.id = 1;
--Retrieve driver details for a vehicle
SELECT d.first_name, d.last_name, d.license_number, d.phone_number
FROM drivers d
JOIN assignments a ON d.id = a.driver_id
WHERE a.vehicle_id = 1;
--Count trips per route
SELECT r.id, r.origin, r.destination, COUNT(a.id) AS trip_count
FROM routes r
LEFT JOIN assignments a ON r.id = a.route_id
GROUP BY r.id, r.origin, r.destination;
```

Output:

SQL ▼ 1 / 1 1 - 1 of 1

make	model	year	license_plate	origin	destination
Toyota	Camry	2020	ABC123	New York	Los Angeles

SQL ▼ 1 / 1 1 - 1 of 1

first_name	last_name	license_number	phone_number
John	Doe	D1234567	555-123-4567

SQL ▼ 1 / 1 1 - 3 of 3

id	origin	destination	trip_count
1	New York	Los Angeles	1
2	Chicago	Houston	1
3	Miami	Atlanta	1

Task 6 – Hotel Reservation Database

Task:

Create a database schema for hotel reservations.

Instructions:

• Tables: Hotels, Rooms, Guests, Reservations.

• Define date constraints and relationships.

• Generate queries:

o List all reservations for a guest.

o Find available rooms for a date range.

o Calculate occupancy rate per hotel.

Expected Output:

• Relational schema and SQL queries for booking analysis.

Code:

```sql
--create hotels table
CREATE TABLE hotels (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(100) NOT NULL,
    address VARCHAR(200) NOT NULL,
    city VARCHAR(50) NOT NULL,
    state VARCHAR(50) NOT NULL,
    zip_code VARCHAR(10) NOT NULL
);
--create rooms table
CREATE TABLE rooms (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    hotel_id INTEGER NOT NULL,
    room_number VARCHAR(10) NOT NULL,
    room_type VARCHAR(20) CHECK(room_type IN ('single', 'double', 'suite')) NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (hotel_id) REFERENCES hotels(id)
);
--crete guests table
CREATE TABLE guests (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone_number VARCHAR(20)
);
--create reservations table
CREATE TABLE reservations (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    guest_id INTEGER NOT NULL,
    room_id INTEGER NOT NULL,
    check_in_date DATE NOT NULL,
    check_out_date DATE NOT NULL,
    total_price DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (guest_id) REFERENCES guests(id),
    FOREIGN KEY (room_id) REFERENCES rooms(id)
);
--Insert sample data into hotels table
INSERT INTO hotels (name, address, city, state, zip_code) VALUES
('Grand Hotel', '123 Main St', 'New York', 'NY', '10001'),
('Ocean View Resort', '456 Beach Ave', 'Miami', 'FL', '33101'),
('Mountain Lodge', '789 Alpine Rd', 'Denver', 'CO', '80201');
```

```sql
--Insert sample data into rooms table
INSERT INTO rooms (hotel_id, room_number, room_type, price) VALUES
(1, '101', 'single', 100.00),
(1, '102', 'double', 150.00),
(2, '201', 'suite', 300.00),
(2, '202', 'double', 200.00),
(3, '301', 'single', 120.00);
--Insert sample data into guests table
INSERT INTO guests (first_name, last_name, email, phone_number) VALUES
('Alice', 'Johnson', 'alice.johnson@example.com', '555-123-4567'),
('Bob', 'Smith', 'bob.smith@example.com', '555-987-6543'),
('Charlie', 'Brown', 'charlie.brown@example.com', '555-555-5555');
--Insert sample data into reservations table
INSERT INTO reservations (guest_id, room_id, check_in_date, check_out_date, total_price) VALUES
(1, 1, '2022-01-01', '2022-01-05', 400.00),
(2, 3, '2022-01-10', '2022-01-15', 1500.00),
(3, 5, '2022-01-20', '2022-01-25', 600.00);
--List all reservations for a guest
SELECT * FROM reservations WHERE guest_id = 1;
--Find available rooms for a date range
SELECT * FROM rooms WHERE id NOT IN (
    SELECT room_id FROM reservations
    WHERE check_in_date <= '2022-01-10' AND check_out_date >= '2022-01-05'
);
--Calculate occupancy rate per hotel
SELECT h.name, COUNT(r.id) AS occupied_rooms, COUNT(ro.id) AS total_rooms,
       (COUNT(r.id) * 100.0 / COUNT(ro.id)) AS occupancy_rate
FROM hotels h
LEFT JOIN rooms ro ON h.id = ro.hotel_id
LEFT JOIN reservations r ON ro.id = r.room_id AND r.check_in_date <= '2022-01-10' AND r.check_out_date >= '2022-01-05'
GROUP BY h.id, h.name;
```

Output:

SQL ▼ 1 / 1 1 - 1 of 1

id	guest_id	room_id	check_in_date	check_out_date	total_price
1	1	1	2022-01-01	2022-01-05	400

SQL ▼ 1 / 1 1 - 3 of 3

id	hotel_id	room_number	room_type	price
2	1	102	double	150
4	2	202	double	200
5	3	301	single	120

SQL ▼ 1 / 1 1 - 3 of 3

name	occupied_rooms	total_rooms	occupancy_rate
Grand Hotel	1	2	50.0
Ocean View Resort	1	2	50.0
Mountain Lodge	0	1	0.0